

CapsuleGuard

Sistema IoT para Monitoramento de Cápsula Espacial

Disciplina	Computational Thinking Using Python
Plataforma de simulação	Tinkercad
Microcontrolador	Arduino UNO
Data de entrega	Junho 2026

1. Introdução

A exploração espacial exige sistemas altamente confiáveis para garantir a segurança dos astronautas e o funcionamento adequado dos módulos espaciais durante missões. Em ambientes extremos, fatores como variações de temperatura, falhas estruturais e alterações de luminosidade precisam ser monitorados continuamente para evitar riscos e garantir condições adequadas de sobrevivência e operação.

Nesse contexto, a Internet das Coisas (IoT) desempenha papel essencial, permitindo a coleta, processamento e exibição de dados em tempo real por meio de sensores integrados a sistemas microcontrolados. O projeto CapsuleGuard propõe o desenvolvimento de uma solução tecnológica para o monitoramento das condições internas de uma cápsula espacial, simulando um cenário próximo às aplicações reais utilizadas em missões aeroespaciais.

2. Objetivo do Projeto

Desenvolver um sistema IoT embarcado capaz de monitorar em tempo real as variáveis físicas essenciais para a operação segura de uma cápsula espacial, com geração automática de alertas quando qualquer parâmetro ultrapassar os limites de segurança estabelecidos.

O sistema monitora as seguintes grandezas:

- **Temperatura interna** — detecta superaquecimento dos sistemas de propulsão
- **Luminosidade** — indica a posição orbital da cápsula (lado claro ou escuro)
- **Vibração** — detecta impactos, turbulências ou falhas mecânicas

3. Componentes Utilizados

Componente	Ambiente	Função	Conexão
Arduino UNO	Físico e Tinkercad	Microcontrolador principal	—
Sensor DHT11	Físico	Temperatura e umidade	Pino digital 2
Sensor TMP36	Tinkercad	Temperatura (substituto do DHT11)	Pino analógico A0
Sensor LDR	Físico e Tinkercad	Luminosidade ambiente	A0 (físico) / A2 (Tinkercad)
Potenciômetro	Físico e Tinkercad	Simulação de vibração	Pino analógico A1
Display LCD 16x2	Tinkercad	Exibição de dados e alertas	Pinos 7, 8, 9, 10, 11, 12
Buzzer	Tinkercad	Alerta sonoro de impacto	Pino digital 6
Resistor 10kΩ	Físico e Tinkercad	Divisor de tensão para LDR	Entre A0/A2 e GND
Resistor 1kΩ	Tinkercad	Contraste fixo do LCD (VO)	Entre VO e GND

Nota sobre limitações do Tinkercad: Por limitação da plataforma Tinkercad, dois componentes foram substituídos por equivalentes disponíveis na simulação. O sensor DHT11 foi substituído pelo TMP36, um sensor analógico de temperatura que dispensa biblioteca externa — em um sistema real o DHT11 seria utilizado conforme implementado no hardware físico. O sensor acelerômetro foi substituído por um potenciômetro para simular leituras de vibração — em produção seria utilizado um MPU-6050 via I2C.

4. Desenvolvimento do Circuito

O circuito foi desenvolvido com base em uma arquitetura de três camadas: camada de sensores, camada de processamento e camada de saída.

Camada de Sensores

No hardware físico, o DHT11 é conectado ao pino digital 2 do Arduino com resistor pull-up de 10kΩ, e o LDR é configurado como divisor de tensão com resistor de 10kΩ em série no pino A0. Na simulação do Tinkercad, por indisponibilidade do DHT11, foi utilizado o sensor TMP36 — um sensor analógico de temperatura conectado ao pino A0, cujo valor é convertido para Celsius pela fórmula: $temp = (tensão - 0.5) * 100$. O LDR foi movido para o pino A2 no Tinkercad para não conflitar com o TMP36. O potenciômetro tem seus terminais externos ligados ao 5V e GND, com o

terminal central no pino A1, em ambos os ambientes.

Camada de Saída — Tinkercad

No Tinkercad foram implementadas duas saídas adicionais: o display LCD 16x2 e o buzzer. O LCD é conectado nos pinos 7 (RS), 8 (EN), 9 (D4), 10 (D5), 11 (D6) e 12 (D7), com o pino VO ligado a um resistor fixo de 1kΩ para contraste, RW conectado ao GND e backlight alimentada pelo 5V. O display alterna automaticamente entre três telas a cada 2 segundos exibindo temperatura, luminosidade e vibração. Em caso de alerta, a tela trava na mensagem de alerta correspondente. O buzzer está conectado ao pino digital 6 e emite som sempre que a vibração indica possível impacto (vibração >= 0.3), silenciando automaticamente quando o valor retorna ao normal.

Camada de Processamento

O Arduino UNO processa as leituras dos três sensores a cada 2 segundos, aplica a lógica de verificação de limites e determina o estado do sistema. As leituras analógicas são convertidas para unidades físicas compreensíveis antes da exibição.

Lógica de Alertas

Variável	Condição	Alerta
Temperatura	>= 25°C	MOTORES MUITO QUENTES
Temperatura	>= 24.5°C	MOTORES AQUECENDO
Vibração	= 1.0 (máx)	IMPACTO DETECTADO
Vibração	>= 0.6	PREPARAR PARA IMPACTO
Vibração	>= 0.3	POSSÍVEL IMPACTO
Luminosidade	<= 100	LADO ESCURO DA ÓRBITA
Luminosidade	< 200 e caindo	ENTRANDO NO LADO ESCURO
Luminosidade	< 200 e subindo	SAINDO DO LADO ESCURO

5. Código Desenvolvido

Foram desenvolvidas duas versões do código: uma para o hardware físico utilizando o sensor DHT11 com a biblioteca DHT, e outra para o Tinkercad utilizando o TMP36 com leitura analógica, além do display LCD 16x2 e buzzer. A seguir são apresentados os trechos principais de cada versão.

Versão Física — Setup e leitura do DHT11:

```
#include <DHT.h> DHT dht(2, DHT11); // sensor DHT11 no pino 2 float luz_ant = 9999.00; // armazena leitura anterior de luz void setup() { Serial.begin(9600); // comunicacao serial para debug dht.begin(); // inicializa sensor DHT11 } void loop() { float vibracao = analogRead(A1) / 1023.00; // 0.0 a 1.0 float temperatura = dht.readTemperature(); // graus Celsius float luz = analogRead(A0); // 0 a 1023 }
```

Versão Tinkercad — Setup com LCD, buzzer e leitura do TMP36:

```
#include <LiquidCrystal.h> LiquidCrystal lcd(7, 8, 9, 10, 11, 12); // pinos do LCD
float luz_ant = 9999.00; void setup() { Serial.begin(9600); lcd.begin(16, 2); //
inicializa LCD 16 colunas, 2 linhas pinMode(6, OUTPUT); // buzzer no pino 6 } void
loop() { float vibracao = analogRead(A1) / 1023.00; float tensao = analogRead(A0) *
5.0 / 1024.0; // TMP36 float temperatura = (tensao - 0.5) * 100.0; // conversao para
Celsius float luz = analogRead(A2); // LDR em A2 }
```

Display LCD — alternância de telas e exibição de alertas:

```
// buzzer toca apenas quando ha vibracao de impacto if (vibracao >= 0.3) { tone(6,
1000, 500); // frequencia 1000Hz por 500ms } else { noTone(6); } // display: alerta
ou tela normal lcd.clear(); if (alerta) { lcd.setCursor(0, 0); lcd.print(msgLinha1);
// ex: "!!IMPACTO!!" lcd.setCursor(0, 1); lcd.print(msgLinha2); // ex: "Vibr: 1.00"
} else { int tela = (millis() / 2000) % 3; // alterna a cada 2 segundos if (tela ==
0) { lcd.print("Temperatura:"); ... } else if (tela == 1) {
lcd.print("Luminosidade:"); ... } else { lcd.print("Vibracao:"); ... } }
```

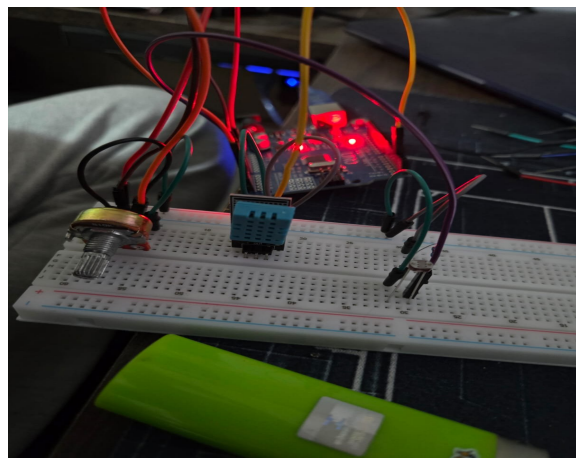
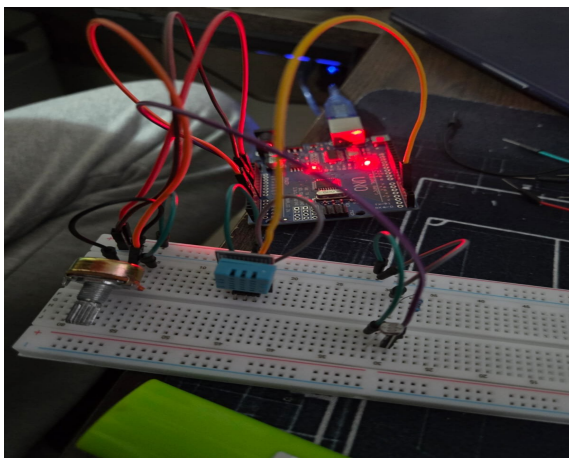
6. Resultados Obtidos

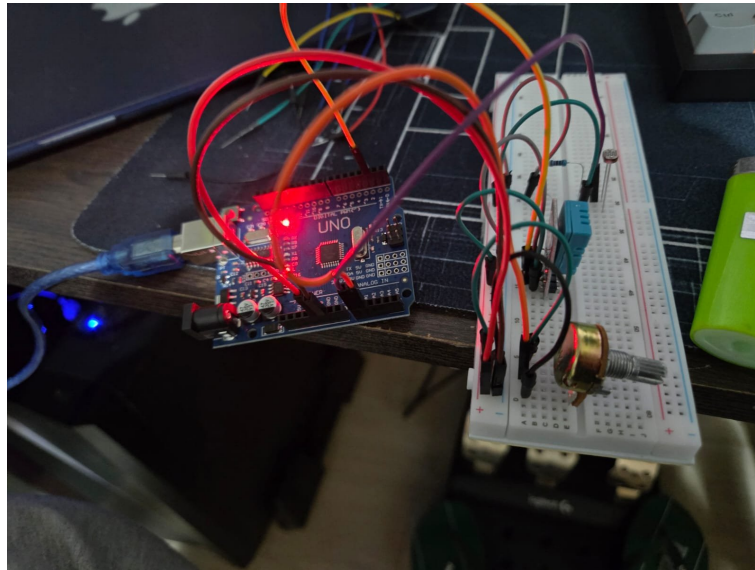
O sistema foi testado tanto na simulação do Tinkercad quanto no hardware físico, com todos os componentes funcionando corretamente:

Hardware Físico:

- O sensor DHT11 retornou leituras estáveis de temperatura ambiente (~23.8°C)
- O LDR detectou variações de luminosidade, identificando corretamente entrada e saída do lado escuro da órbita
- O potenciômetro variou corretamente entre 0 e 1023 simulando níveis de vibração
- O Serial Monitor exibiu mensagens de status e alerta em tempo real a cada 2 segundos

Fotos do circuito físico:





Simulação Tinkercad:

- O sensor TMP36 retornou leituras corretas de temperatura após conversão analógica
- O display LCD 16x2 alternou corretamente entre as três telas de monitoramento
- Em situação de alerta, o LCD travou na mensagem correspondente com clareza
- O buzzer soou corretamente ao detectar vibração acima do limite, silenciando ao normalizar
- A lógica de rastreamento orbital (luz_ant) funcionou corretamente para detectar a direção da variação de luz

7. Melhorias Futuras

- **Substituir o potenciômetro pelo MPU-6050** — acelerômetro real para medição precisa de vibração via I2C
- **Implementar LCD e buzzer no hardware físico** — replicar a experiência completa do Tinkercad no circuito físico
- **Frequências diferentes no buzzer** — tom específico para cada tipo de alerta (temperatura, vibração, luminosidade)
- **Comunicação WiFi via ESP32** — envio dos dados para dashboard web em tempo real
- **Armazenamento em cartão SD** — log de todas as leituras para análise pós-missão
- **Sensor de pressão BMP280** — monitoramento da pressão interna da cápsula

8. Conclusão

O projeto CapsuleGuard demonstrou a viabilidade do uso de sistemas embarcados IoT para monitoramento de variáveis físicas críticas em ambientes espaciais. A solução foi desenvolvida em dois ambientes complementares: o hardware físico com DHT11, LDR e potenciômetro para validação real dos sensores, e a simulação no Tinkercad com TMP36, LDR, potenciômetro, display

LCD 16x2 e buzzer para demonstração completa do sistema com interface visual e sonora.

Os principais aprendizados incluem: programação de microcontroladores Arduino, uso de bibliotecas externas (DHT.h e LiquidCrystal.h), leitura de sensores analógicos e digitais, lógica condicional para sistemas de alerta, controle de display LCD, geração de som com buzzer via `tone()`, e a importância de variáveis de estado (`luz_ant`) para rastreamento de tendências ao longo do tempo. O projeto estabelece uma base sólida para evoluções futuras com comunicação wireless e dashboard de monitoramento remoto.

Links do Projeto

Recurso	Link
Simulação Digital (Tinkercad)	https://www.tinkercad.com/things/i7OMs88fR82-copy-of-gs-computer-organization
Simulação Físico (Tinkercad)	https://www.tinkercad.com/things/bYJ9lx8977m-fisico-gs-computer-organization
Código Fonte (GitHub)	Disponível no repositório do projeto
Vídeo Explicativo (YouTube)	https://youtube.com/watch?v=tXIYlk_wyR4